Predicting Dropout Risk of First-Year College Students Using Socioeconomic, Demographic, and Academic Performance Metrics

Sarah Gillard

January 19, 2026

Table of Contents

Executive Summary *1*

Project Proposal Overview *3*

Real-World Business Problem **3**
Business Problem Importance 3
Optimal Solution 4

Customers and Their Needs **5**
Meeting Customers' Needs 5

Project Stakeholders **6**
Meeting Project Stakeholders' Needs 6

Gaps Between the Current System and the Organization's Needs **7**

Project Goals **7**

How Others Have Addressed the Business Problems **8**
Business Problem Importance 9

Project Plan *10*

Development Approach and Project Methodology **10**

Project Phases **12**

Project Schedule **15**

Project Resources **18**

Financial Breakdown **20**

Implementation Plan **23**

Training and Support Plan **26**

Project Risks **27**
Risk Mitigation Strategy 28

Evaluation Methods and Success Methods **29**

Postlaunch Performance Monitoring Plan **29**

Continuous Improvement Plan **30**

Technical Report *31*

Product Functionalities and Capabilities **31**

System Architecture **31**

Functional Requirements **32**

Nonfunctional Requirements **33**

Flowchart and Use Case Diagram **34**

Data Handling **36**

Algorithm Functionality **38**

Product Integration **41**

Privacy, Security, and Compliance Requirements **41**

Acceptance Criteria **41**

Testing Approach **42**

Test Case Examples **43**

Product Improvement or Enhancement **45**

Proposal Summary ***46***

Conclusion **46**

Link to GitLab ***47***

Links to Google Colab notebooks ***47***

References ***48***

Executive Summary

According to the Education Data Initiative, most college students drop out during their first academic year (Hanson, M., 2025). Student attrition presents several challenges for universities. These challenges include declining graduation rates, a damaged reputation, and financial instability. Early recognition of students who are at risk of dropping out is essential for thwarting attrition, improving graduation rates, and supporting the university's public image and financial strength. The proposed machine learning project offers a solution to help administrators at higher education institutions identify first-year college students who are at an increased risk of dropping out, as well as predict which first-year students are likely to graduate and remain enrolled. The project utilizes demographic, socioeconomic, and academic performance data collected from over 4,000 students to train a gradient boosting classifier model, which makes predictions about outcomes for first-year college students. This system will enable universities to target students who are most at risk of dropping out, providing them with targeted intervention supports and making informed decisions based on data to enhance student success and improve the university's public perception of its academic performance.

Exploratory data analysis will be performed on the dataset using a multiple linear regression model and various visuals to examine feature importance and inform the final product design. A gradient boosting model with stochastic subsampling will be developed and trained on the dataset. The gradient boosting model should achieve a classification prediction accuracy rate of 75% or higher in predicting which first-year students will drop out, remain enrolled, or graduate. The exploratory data analysis will facilitate the identification of which features to include in the

user interface as input, allowing for the best prediction without the need to include all 34 feature variables provided by the dataset.

A prototype of the gradient boosting model will be trained and evaluated using classification metrics, including accuracy, precision, recall, and F1-score. The baseline model is planned to be trained, tested, validated, evaluated, and then executed with a user interface simulation to demonstrate practicality and usability to administrators. Feedback will be reviewed and incorporated throughout the development process until the final model is successfully completed. With the proposed solution implemented, universities will better identify students who are at risk of dropping out during their first academic year. This allows support staff to adapt and improve their retention strategies to provide enhanced supports that raise academic outcomes for at-risk students. The project anticipates the completion of a machine learning model to enhance and improve university performance and student success. The project will be developed using the Agile methodology, as flexibility and continuous feedback are essential in ascertaining key features to include in the user interface.

The product will be developed using an Agile development approach with the Scrum framework, which breaks tasks into sprints with feedback incorporated at the end of each sprint through a Sprint Review. The project is expected to take approximately six to eight weeks to complete. The first week will focus on planning and requirements, establishing the business objective, identifying stakeholders, defining success metrics, and establishing a Definition of Done. Additionally, preprocessing the dataset and performing exploratory data analysis will be conducted. The remaining five to seven weeks will include two or three sprints for feature selection, as well as developing, training, and testing the gradient boosting classifier model for

prediction and usability of the UI. Each sprint, improvements will be made based on user feedback gathered during the demo phase. The final week will focus on creating documentation for user training and launching the completed prediction model product.

Project Proposal Overview

Real-World Business Problem

According to the Education Data Initiative, most college students who do not complete their degree program drop out during their first year of academic study (Hanson, M., 2025). Universities are challenged with identifying which first-year students are most at risk of dropping out early in their academic program. Without accurate identification of these students made in time, institutions struggle to offer effective interventions and assistance that may lead to retention and prevent an increasing attrition rate.

Higher education institutions collect student demographic, socioeconomic, and academic performance data, but many do not use this information effectively to inform proactive approaches to retention. Without accurate predictive tools, administrators struggle to identify at-risk students early and provide targeted academic, financial, and emotional support that may preserve these students beyond the first academic year. Many students are not engaged with their schooling and drop out before the interventions are brought to their attention.

Business Problem Importance

Student attrition at the university level has negative academic, financial, and societal consequences. The economic effects of student attrition at the college level include a decrease in

the purchasing power of students who drop out, which in turn leads to a decline in GDP (gross domestic product), as well as a reduction in income potential for students who do not graduate (Guzman et al., 2021). High student attrition can lead to reduced workforce productivity, which in turn can result in decreased economic growth.

When higher education institutions can effectively analyze student data and identify which students are most at risk of dropping out, they can provide targeted interventions and support promptly. These supports often lead to increased graduation rates (Apumayta et al., 2024), promoting student success that reflects positively on the university and provides a more educated population, thereby benefiting both individual students and the greater society.

Optimal Solution

By providing a simple, user-friendly interface that predicts a first-year student's academic outcome, the proposed solution addresses retention issues faced by universities, as it will predict if a student is likely to drop out, remain enrolled, or graduate after their first academic year. These predictions allow administrators to provide timely, targeted interventions that may alter dropout predictions by retaining those students. The solution is built using a gradient boosting classifier model trained on common institutional data (demographic, socioeconomic, and academic performance), so no additional data collection is needed from the client to initiate project development.

The user interface provides a prediction that is interpretable and usable by non-technical workers. The model balances optimization of hyperparameter values identified during model tuning to achieve strong predictive accuracy. The model is scalable, allowing for the

incorporation of more student data over time, which can further improve its performance through consistent training on new data. The model is also low-risk, as predicting student dropout risk is important but not life-critical, unlike predictive models used in the healthcare realm, which require much higher accuracy before deployment.

A multiple linear regression model will be used for exploratory data analysis to identify the key socioeconomic, demographic, and academic performance factors with the most significant influence on student success outcomes. Visualizations, such as correlation heatmaps, will be provided to inform administrators about the relationships between the collected student data, enabling them to understand and act on the variables with the highest correlations to student retention and success.

Customers and Their Needs

The target customers of the proposed AI/ML product are university administrators, academic advisors, and individuals working on teams responsible for supporting first-year college students. These users benefit from data-driven methods to identify students who are at the most significant risk of dropping out during their first academic year. The customers' needs include early risk identification and understandable predictions from an intelligible product to assist in identifying at-risk students, thereby providing supports that improve student retention and graduation outcomes.

Meeting Customers' Needs

The proposed technical solution meets the customer's needs by providing an intuitive user interface that enables non-technical administrators and staff to input essential student

information, which the model uses to produce a precise prediction of student graduation outcomes. This prediction can inform intervention initiatives aimed at improving student retention.

Project Stakeholders

Key stakeholders in the project include university administration, faculty, academic advising members, student service support staff, research teams, and enrolled students. University administration will benefit from improving retention metrics, which will lead to better financial stability and a stronger reputation. Academic advisors and student service support staff can use this model to provide data-driven insights to aid in their intervention efforts. Students are stakeholders, albeit indirectly, as the prediction interface is designed to improve timely student support offerings which benefits first-year students. All stakeholders are affected by student retention, and the proposed solution will benefit all parties.

Meeting Project Stakeholders' Needs

The technical solution proposed meets the project stakeholders' needs by providing an interpretable, data-driven product that supports the early identification of students at risk of dropping out in their first year. University administrations will gain actionable insights to implement data-driven retention initiatives and resources. Academic advising teams and student support staff benefit from having data-driven predictions provided promptly, which informs their interventions and reduces errors involving academic, financial, and personal support services to students. Researchers can utilize the model predictions to gain a deeper understanding of trends

and factors influencing student retention. Students benefit from being provided with early interventions and supports that increase their chances of academic success.

Gaps Between the Current System and the Organization's Needs

Many universities lack practical approaches to identifying at-risk students in a timely and data-driven manner. Some professionals may rely on manual approaches to identify at-risk students through grades and attendance records. However, these methods do not focus on the most important factors that affect student retention in a scalable, reliable, and tested manner. This prevents facilities from providing fast, targeted interventions to students at risk of dropping out in their first year. The proposed solution will bridge the gap by using a dataset of demographic, socioeconomic, and academic performance data to calculate a prediction of student success outcomes with an accuracy rate of approximately 75-80%. The interactive user interface will enable staff to enter just six key variables that most affect student dropout risk and calculate a prediction of dropout, remaining enrolled, or graduating, with an accuracy rate close to 80%.

Project Goals

1. The proposed solution will develop a gradient boosting classifier model that predicts whether a first-year college student will drop out, remain enrolled, or graduate, with an overall classification accuracy of approximately 75% or higher.

2. A user interface will be designed and implemented in simulations to collect feedback. Then a final UI will be developed that allows university staff of all technical backgrounds to input six key student attributes and receive a prediction outcome.

How Others Have Addressed the Business Problems

Prior research has demonstrated that machine learning models can be practical tools for predicting student performance and identifying students at risk of poor academic outcomes. Farhood et al. (2024) evaluated multiple machine learning and deep learning models to predict whether students passed or failed their final exams. Their study emphasized the importance of having a structured workflow that includes data preprocessing, feature selection, hyperparameter tuning, and cross-validation. They found that an ensemble learning model outperformed simpler classifier models, with random forest and XGBoost performing the best. Their results also demonstrated that k-fold cross-validation improved predictive accuracy across models compared to basic holdout evaluation. These findings support the use of ensemble learning methods and cross-validation techniques in educational prediction models, which aligns with the proposed project's use of a gradient boosting classifier with stratified cross-validation to improve prediction reliability.

Ab Rahman et al. (2025) further reinforced the effectiveness of ensemble learning approaches to student success through a comprehensive review of literature focused on educational data analysis for student outcomes. Their study highlighted that ensemble models, including gradient boosting, achieve higher prediction accuracy than individual classifier models, especially when paired with effective feature selection techniques. The authors emphasized that reducing model complexity by selecting the most relevant features improves scalability and interpretability while maintaining strong performance. Their findings support the design choice in the proposed solution to limit user inputs to a small set of highly correlated features identified through exploratory data analysis, rather than requiring all variables to be input by the user.

Villar and Velini de Andrade (2024) conducted a study comparing supervised machine learning models for predicting student dropout, enrollment, and graduation outcomes. Their work showed that models incorporating gradient boosting performed better in multi-class classification problems involving imbalanced educational datasets. While they applied techniques such as SMOTE to address class imbalance and they explored the Optuna optimization framework, their results showed that gradient boosting models maintained high performance even without Optuna. The authors noted that extreme gradient boosting provided marginal improvements in accuracy at the cost of increased computational complexity. These findings support the proposed project's decision to use a gradient boosting classifier model with stochastic subsampling to balance predictive performance with computational simplicity.

These studies demonstrate that ensemble learning methods, such as gradient boosting, are well-suited for predicting student success outcomes. They also highlight the importance of cross-validation, feature selection, and controlled model complexity during model selection. These previous studies support the proposed solution's technical approach and reinforce its potential effectiveness as a data-driven tool for identifying at-risk first-year college students.

Business Problem Importance

Student attrition at the university level has negative academic, financial, and societal consequences. The economic effects of student attrition at the college level include a decrease in the purchasing power of students who drop out, which in turn leads to a decline in GDP (gross domestic product) as well as a reduction in income potential for students who do not graduate . High student dropout rates can lead to reduced workforce productivity, which in turn can result in decreased economic growth (Guzman et al., 2021).

When higher education institutions can effectively analyze student data and identify which students are most at risk of dropping out, they can provide targeted interventions and support promptly. These supports often lead to increased graduation rates (Apumayta et al., 2024), promoting student success that reflects positively on the university and provides a more educated population, thereby benefiting both individual students and the greater society.

Project Plan

Development Approach and Project Methodology

The product will be developed using the Agile methodology with a Scrum framework. This approach to project management works well as creating the proposed project requires flexibility, iterative development, and continuous fine-tuning based on customer and stakeholder feedback. Feature selection, model tuning, and designing a user-friendly interface require flexibility to adapt the product through model demonstrations with stakeholders and exploratory data analysis.

Agile development that uses the Scrum framework involves short, iterative sprints in which the model and user interface are improved, based on feedback, until a final product that satisfies the customers' Definition of Done is reached. As Paul VII explains in his Scrum certification audiobook, the Definition of Done states the conditions and quality measures that must be reached for the product to be considered complete and ready for deployment (Paul VII, 2017). Throughout the development process, the model will be trained, evaluated, and adapted using performance metrics such as accuracy, precision, recall, and F1-score. Feedback from model demonstrations will help guide the team in making improvements in areas such as feature selection, hyperparameter tuning, and user interface design.

The Agile methodology implemented with the Scrum framework reduces risks, as issues can be addressed as they arise, rather than waiting until a final product deployment to identify bugs (which happens when using the rigid Waterfall design approach). Testing, validation, and feedback are continuous throughout product development, rather than occurring in rigid steps, as in the Waterfall methodology. While the Agile methodology may not be known for providing detailed documentation, the team will provide code comments, a README file, and clear instructions for the user. These requirements will support future development teams working on the product, as well as customers who will use it.

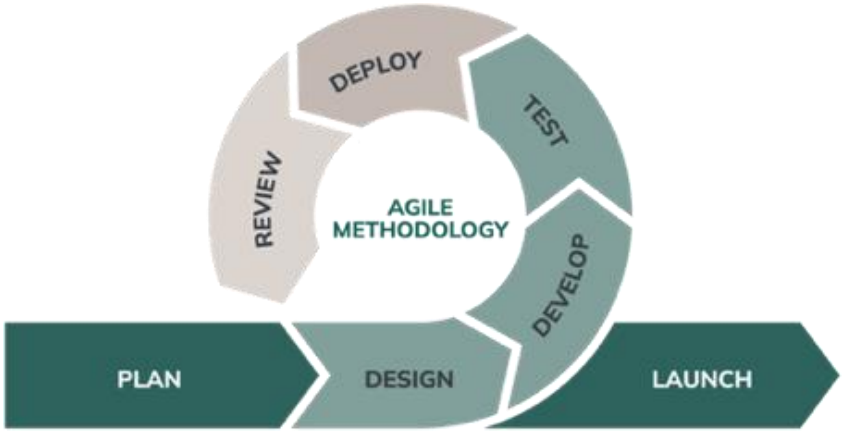

Figure 1: The Agile Methodology Phases

Note. From *Agile software development: Everything you need to know* [Image], by P.-J. L. Michaud, 2024, Nexapp (https://www.nexapp.ca/en/blog/agile-software-development).

Project Phases

The team will hold daily scrum meetings to share progress, successes, and challenges related to their current tasks, and the Scrum Master (project manager) will record this information to inform timeline estimates, which may change due to the flexibility that the Agile methodology provides. However, the timeline should not exceed the maximum 8-week deadline that includes buffer time, as the customer is expecting a product they define as "done" by the end of eight weeks.

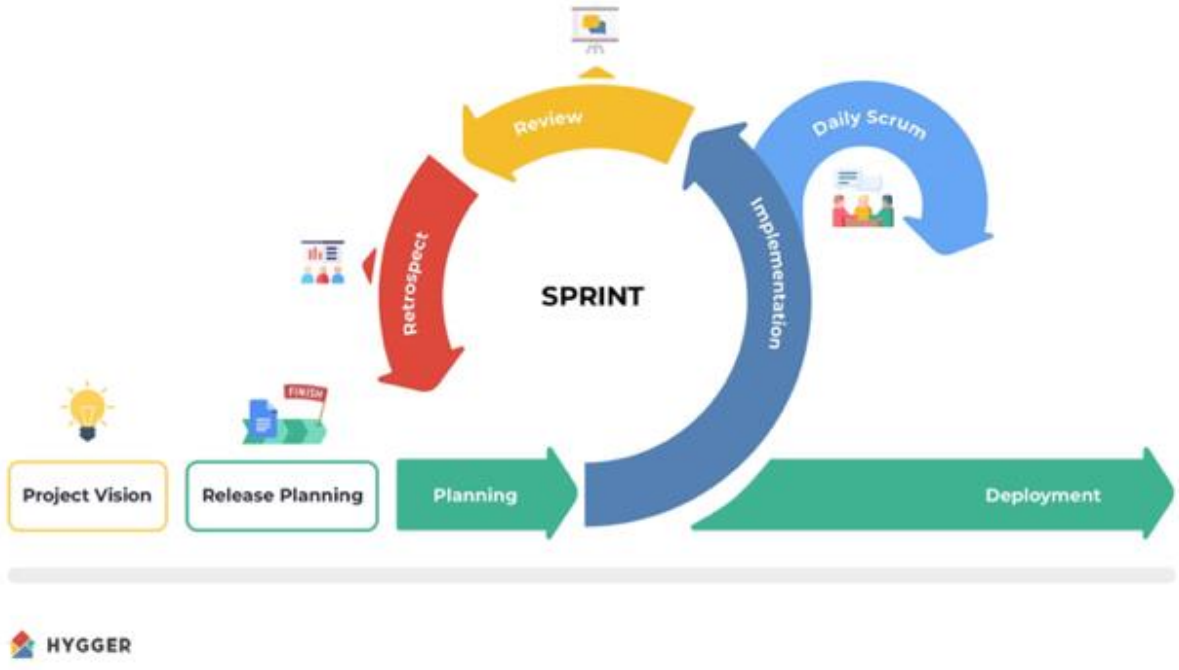

Figure 2: Scrum Sprint

Note. From *Sprints: All you need to know about sprints in project management* [Image], by P. Kukhnavets, n.d., Hygger (https://hygger.io/guides/agile/scrum/sprints-in-scrum/).

Phases two through four are executed in an iterative process known as an "Agile sprint," or just "a sprint" when following the Scrum framework. Each sprint has a goal of producing a completed model to demonstrate to stakeholders with the objective of meeting the customer's Definition of Done. Each sprint includes design, development, testing, demo deployment, and review tasks. Each sprint concludes with a formal team meeting to review feedback and determine whether the product is ready for launch or if another sprint is necessary. Multiple Agile sprints may be executed until the stakeholder requirements and Definition of Done are met, at which point the launch phase commences.

Phase 1- Project Vision, Release Planning, Planning:

- The business objective for the project will be defined through collaboration between the product stakeholders and the data analytics team. This objective will be documented by the Scrum Master.
- The team will identify key performance indicator (KPI) metrics for assessing the product's performance, and a Definition of Done will be established for the final product.
- The dataset will be preprocessed through data cleaning and transformation, including scaling numerical data and encoding the target variable.

Phase 2- Implementation (Includes design, development, and testing):

Design:

- The development team will focus on designing the gradient boosting model and the user interface based on the stakeholder needs obtained in Phase 1.

- Feature selection will be conducted, and a baseline gradient boosting classifier model with a prototype of a user interface will be built.

Development:

- The numerical and categorical features will be scaled, and the model will be trained in an 80/20 test/train split.
- A user interface prototype will be developed to demonstrate the predictive capabilities of the model. This UI will be implemented during the demo with stakeholders, in which feedback on how to improve it will be collected.

Testing:

- During the testing phase, model evaluation and validation will be conducted by the data analytics team for quality assurance using the key performance indicator metrics featured in the product's Definition of Done.

Phase 3- Sprint Review:

- A product demo will be deployed to stakeholders, including the customer, to gather feedback on the model's performance and usability for the development team to use to improve the model during the next sprint.

Phase 4- Sprint Retrospective:

- The feedback collected during the testing phase will be analyzed during a daily team meeting. The team will use the feedback collected from the demo with stakeholders to refine features and the user interface design, as well as retrain the model, if necessary.

Phases two, three, four, and five will be iterated upon in another sprint until the customer is satisfied with the product and ready to proceed to the final phase.

Phase 5- Launch:

- The development team will prepare documentation with user instructions for use in training customers on the final product.
- The final product will be deployed for customer use during the launch phase.

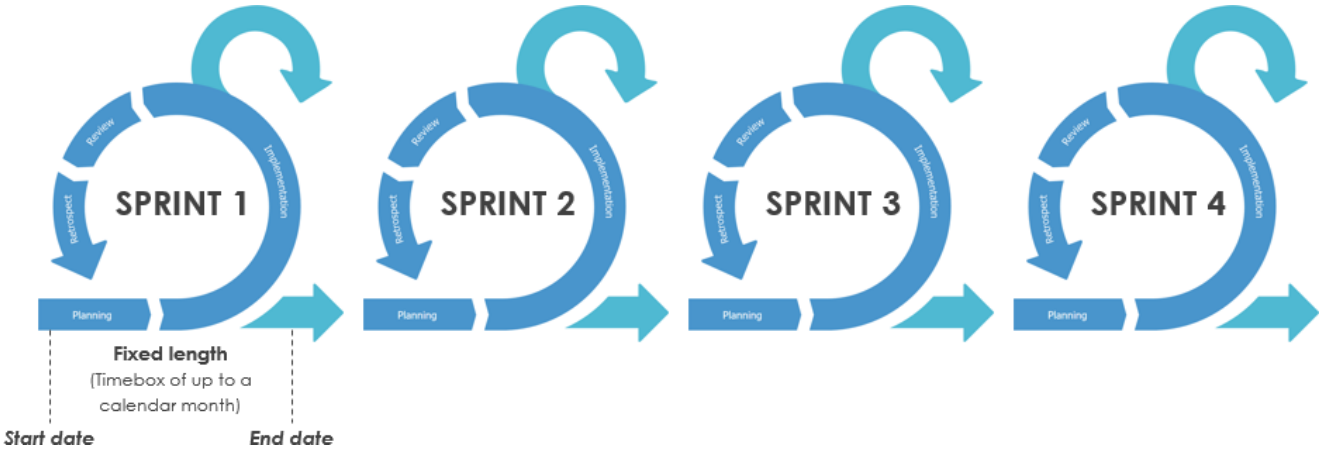

Figure 3: Sprints in Scrum

Note. From *Why fixed length sprints in scrum?* [Image], no author, 2025. Visual Paradigm (https://www.visual-paradigm.com/scrum/why-fixed-length-of-sprints-in-scrum/).

Project Schedule

A schedule is provided for the project with estimated dates, as Agile is a flexible design process that values adapting to feedback over rigid deadlines. Applying the Scrum framework to the Agile methodology breaks the project into sprints that focus on creating a model to demo throughout an implementation process, to gain feedback that will be used in the next sprint until the Definition of Done is reached and product launch ensues.

Project Phase/Sprint	Duration	Focus	Start Date (Tentative)	End Date (Tentative)
Phase 1- **Project Vision, Release Planning, Planning:**	1 week	Define business objectives, identify stakeholders, establish KPI metrics and a Definition of Done, and preprocess the dataset.	February 2, 2026	February 6, 2026
Sprint 1- **Iteration 1 of Phases 2-4**	1.5 weeks	Design a baseline model with a user interface, develop the model, conduct model testing, present the demo to stakeholders, and review feedback.	February 9, 2026	February 18, 2026
Sprint 2- **Iteration 2 of Phases 2-4**	1.5 weeks	Feature refinement, model tuning, model retraining, feedback-driven UI improvements, model testing, demo to stakeholders, and review feedback.	February 18, 2026	February 27, 2026
Sprint 3- **Iteration 3 of Phases 2-4**	1.5 weeks	Performance tuning of the model, model validation, and improvements for user	March 2, 2026	March 11, 2026

		experience, model testing, demo to stakeholders, review feedback. Ask: Has the Definition of Done been reached?		
Sprint 4- ***Iteration 4 of Phases 2-4***	• *After Sprint 3, if the customer is ready for the product to launch, proceed to Phase 5: Launch.* ***It is anticipated that the team should reach the Definition of Done after 3 sprints.*** • *Additional sprints may be implemented if improvements are requested by the customer before launching the product, so an additional* ***1.5 weeks of buffer time*** *has been added to the schedule.* • ***If there is a need for a Sprint 4****, this buffer time would have anticipated dates where Sprint 4 starts on March 11, 2026, and ends on March 20, 2026.* • *If a Sprint 4 is required, Phase 5 will have a start date of March 23, 2026, and an end date of March 27, 2026.*			
Phase 5- **Launch**	1 week	Official deployment of product, user documentation and training materials are created and distributed to customers.	March 11, 2026 *(March 23, 2026, if a Sprint 4 is required.)*	March 18, 2026 *(March 27, 2026, if a Sprint 4 is required.)*

Total Duration:	Anticipated End Date:	Deadline:
6-8 weeks	March 11, 2026	March 27, 2026

Project Resources

The following are required resources for the project:

Human Resources

- **Machine learning engineer (2)**- will develop, train, evaluate, and tune the gradient boosting classifier model.
- **Data analyst** – will perform exploratory data analysis, preprocess, evaluate, and validate data.
- **Software developer** – will design and implement the user interface and integrate it with the machine learning model.
- **Scrum Master (project manager)**- will be responsible for stakeholder communication, sprint planning and deadlines, conducting daily scrum meetings and post-demo retrospectives.
- **University employees (administrators, student support staff)**- these stakeholders will provide domain knowledge, feedback, and validation during product demos and sprint review meetings.

Technology Resources

- **Cloud computing environment**- will support model development and training.
- **Data storage**- will store datasets, trained models, and model evaluation data.

- **Approximately 10 computers (for engineers, analysts, developers, Scrum Master, and Stakeholders)**- will be used for programming a user interface and model, data analysis, and model training and testing.

Tools and Software Resources

- **Python**- will be the programming language used throughout the project.
- **Scikit-learn**- this Python library will be used for data analysis, machine learning model development, and model evaluation.
- **Pandas**- this Python library will used for data preprocessing and manipulation.
- **NumPy**- this Python library will be used for working mathematically with data stored as arrays, as well as for preprocessing and manipulating data.
- **Jupyter Notebooks and Google Colab**- these development environments will be used for data analysis and machine learning.
- **GitHub**- this will be used for dataset integration in Google Colab.
- **Matplotlib**- this Python library will produce data visualizations for model evaluation and exploratory data analysis.
- **Seaborn**- this Python library will produce data visualizations for model evaluation and exploratory data analysis.
- **SciPy**- this statistics-based Python library will be used for randomized hyperparameter search during model tuning.
- **ipywidgets**- this Python library will be used to create an interactive user interface for entering student data and displaying a prediction result to the user.

Financial Breakdown

The financial breakdown includes development, implementation, and maintenance costs for the proposed machine learning product. Costs are estimated and based on an Agile project timeline of approximately 6-8 weeks.

Financial breakdown of development (human resources) costs:

Resource	Role	Hourly Rate	Estimated Cost
Machine learning engineers (2)	Model development, training, model evaluation, and hyperparameter tuning.	$65/hour	41,600 total ($20,800 each for 8 weeks)
Data analyst	Preprocess data, analyze data, validate data, and perform exploratory data analysis.	$50/hour	$16,000 (for 8 weeks)
Software developer	Create and integrate a user interface for the model.	$42/hour	$13,440 (for 8 weeks)
Scrum Master (project manager)	Facilitate and manage stakeholder communication, plan sprints and deadlines,	$62/hour	$19,840 (for 8 weeks)

	conduct daily Scrum meetings, and post-demo retrospectives.		
Total estimated cost of development: $76,480			

Financial breakdown of technology (implementation) costs:

Resource	Use	Estimated Cost
Cloud computing environment	Model training, development, and demos.	$1,500 (for 8 weeks)
Data storage	Storage for datasets, trained models, and model evaluation data.	$500 (for 8 weeks)
Approximately 10 computers	Used by the development team (engineers, analysts, developers, Scrum Master) and Stakeholders.	$7,500 (for 8 weeks) *University hardware should be used for testing, demo, and deployment, but the development team will need equipment capable of performing machine learning at high speed with sufficient RAM and GPU storage.*
Total estimated cost of implementation: $9,500		

Financial breakdown of software and tools (maintenance) costs:

Resource	Use	Estimated Cost
Python	Programming language	$0
Python libraries (scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, SciPy, ipywidgets)	Model development, data preprocessing and manipulation, EDA, statistics for hyperparameter tuning, data visualization, and user interface integration.	$0
Approximately 10 computers	Used by the development team (engineers, analysts, developers, Scrum Master), and Stakeholders.	$0
Jupyter Notebooks and Google Colab	Development environment for data analysis and machine learning.	$0
GitHub	Dataset hosting and integration.	$0
Total estimated cost of maintenance: $0		

Overall Estimated Cost of Project:

Domain	Estimated Cost
Development	$76,480
Implementation	$9,500
Maintenance	$0
8-week Project Total: $85,980	
Estimated yearly maintenance cost	*$12,897*

Implementation Plan

The solution will be implemented using an Agile development approach with the Scrum framework.

1. **Business understanding:**
   - Establish a clear business objective for the project, focused on identifying first-year college students at risk of dropping out.
   - Stakeholders will collaborate with the data analyst and machine learning engineers during Phase 1 (project vision, release planning, and planning) to outline project goals, success metrics, and key performance indicators, and to establish a Definition of Done.

2. **Data understanding:**
   - The dataset will be reviewed to understand the available demographic, socioeconomic, and academic performance feature variables obtained from first-year college students.
   - Exploratory data analysis will be carried out to evaluate feature relevance, handle any potential missing values and data quality issues, and access class distribution of the target variable.
3. **Data preparation:**
   - The dataset will be preprocessed using data cleaning and transformation techniques.
   - The target variable will be encoded for classification, where 0 = drop out, 1 = remain enrolled, and 2 = graduate.
   - Numerical and categorical features will be scaled to be used in training the prediction model.
4. **Modeling:**
   - During the implementation phase, a gradient boosting classifier model using stochastic subsampling will be developed.
   - The model will be trained using an 80/20 test/train split.
   - Hyperparameters will be tuned using stratified k-fold cross-validation and randomized search cross-validation.
   - The modeling activities will repeat each sprint to improve the model for optimized performance, based on stakeholder feedback gathered during the deployment phase.

5. **Evaluation:**
    - During the testing phase of each sprint, the model will be evaluated using classification metrics such as accuracy, precision, recall, and F1-score.
    - Confusion matrices and classification reports will be reviewed to validate that the model meets the delineated success criteria.
    - During sprint review meetings, evaluation results will be presented to stakeholders to validate that the metrics align with the business objective and Definition of Done.
6. **Deployment:**
    - The final model will be integrated with a user interface that allows users to enter key student data to generate a predicted outcome: graduate, remain enrolled, or drop out.
    - Documentation and user instructions will be created to train users on how to ethically use and interpret the model.
    - The solution will be deployed to stakeholders in a demo, which is a simulated production environment, to collect feedback.
    - The deployment phase will repeat at the end of each sprint until the Definition of Done is met. Once it is met, the project will be launched (phase 5).
7. **Maintenance and support:**
    - After the solution is deployed, it will be monitored for performance, accuracy, and usability.
    - Stakeholder feedback will be collected to identify areas for improvement.
    - The model will retrain periodically as new student data becomes available, enabling it to scale to current data trends.

- Ongoing support will be provided to customers by maintaining up-to-date documentation, providing user training, monitoring model accuracy, and implementing other improvements based on user feedback and model KPIs.

Training and Support Plan

During the deployment phase of each sprint, stakeholders will participate in live demonstrations of the user interface. Users will observe how student data is entered into the solution, how predictions are generated, and how to interpret the findings. The development team will create user documentation that provides concise instructions for using the model interface, interpreting prediction outcomes, and understanding limitations. During the implementation of the technical solution, users will receive training and support on using the prediction model. Stakeholders will receive this documentation before the product launches to provide feedback, enabling the development team to adjust it accordingly.

Ongoing support for the product will be provided through feedback meetings with stakeholders, where stakeholders can share questions, concerns, and requests to change the model to reach solutions. After the product launches, support will focus on monitoring model performance, answering user questions, and fine-tuning the user interface and documentation to improve customer usability.

Project Risks

1. **Model bias:**

- The model may introduce bias in the collected student data, potentially affecting predictions for certain demographic or socioeconomic groups. Even if bias is not yet revealed in the initial dataset, it could be introduced later as the model scales to add new student data as some socioeconomic and demographic groups may be included more or less frequently in the dataset.

2. **Prediction accuracy:**

- The model may not achieve sufficient accuracy to reliably identify all first-year college students at risk of dropping out. A student may be predicted to graduate and end up dropping out, for example. The model is just a tool to help target first-year students who may be most in need of support.

3. **User misinterpretation of predicted output:**

- Users may misunderstand the model's predictions, treating them as definitive outcomes for the student rather than as data-driven insights to support students who may not otherwise be reached with retention-aimed resources.

Risk Mitigation Strategy

1. **Mitigating model bias:**

- During exploratory data analysis, model bias will be considered to pinpoint potential imbalances in the dataset.
- Evaluation metrics will be analyzed to ensure the model's predictions are not overly erroneous for specific populations. These metrics will include precision, recall, F1-score, and correlation matrices.
- Any stakeholder feedback about potential bias in the dataset will be documented and used to identify potential bias to abate.

2. **Mitigating risks to prediction accuracy:**

- To respect prediction accuracy, model tuning, feature refinement, cross-validation, and hyperparameter tuning will be performed as needed. Improvements will be based on current performance metrics and stakeholder requirements.

3. **Mitigating user misinterpretation:**

- To help customers navigate the system effectively, clear documentation and training materials will be provided. The user interface will be intuitive to interpret and use, even by non-technical users.
- Deciphering output predictions should be discussed in professional development training sessions with stakeholders to warrant their ethical use.

Evaluation Methods and Success Methods

To assess whether the solution meets the business goals and the Definition of Done, the model will be evaluated using key performance indicator metrics, and feedback will be obtained from stakeholders during each sprint deployment phase. The primary success metric for the model is its classification accuracy for user input. The final gradient boosting classifier must achieve an overall classification accuracy of 75%-80% on the test data.

Model performance will be evaluated using metrics such as accuracy, precision, recall, and F1-score. These values demonstrate how well the model predicts each student outcome category and confirm it is balanced in performance across the three target variable classes. A classification report will also be provided to validate and cultivate feature selection, ensuring that variables with the highest correlation with the target variable are selected for use in the user interface.

Postlaunch Performance Monitoring Plan

Post-launch, the model's performance will be monitored to confirm it continues to meet the business goals and criteria defined in the Definition of Done. Model performance will be validated using KPIs such as accuracy, precision, recall, f1-score, and confusion matrices. When available, the model's predicted outcomes will be compared with the university's updated enrollment and graduation data for the year. This will help identify model performance and data patterns that could affect prediction accuracy. The development team will review feedback from administrators and other stakeholders to assess the model's sustainability and the interpretation of its predictions. When model performance declines, feature refinement and model retraining will be performed.

Continuous Improvement Plan

A continuous improvement plan based on model performance results and stakeholder feedback will be implemented to enhance the proposed solution. User feedback collected after deployment will provide insights for feature selection, model tuning, and user interface design improvements. As new student data becomes available, the model will retrain periodically to reflect current data trends and maintain the accuracy of its predictions. Any updates to the system will be evaluated against the defined success metrics to confirm the solution remains aligned with the customer's goals for identifying students at risk of dropping out during their first academic year.

Technical Report

Product Functionalities and Capabilities

The proposed product is designed to help higher education administrators identify first-year college students who are at risk of dropping out, likely to remain enrolled, or likely to graduate. The predictive system allows users to input a limited set of key student data points through a simple, interactive user interface. The user will be able to review a predicted outcome generated by a trained gradient boosting classifier, based on their input values.

The primary capabilities of the product include transforming user input data to scaled values, generating classification predictions, and presenting the results in a clear format for non-technical users to interpret. The project supports exploratory data analysis, model evaluation, and iterative improvement based on frequent feedback. The proposed product's functionality enables the system to support movers and shakers in creating data-informed retention initiatives at universities in an accessible format that does not require advanced mathematical or technical knowledge from users.

System Architecture

The system architecture follows a cohesive design plan encompassing data processing, model training and prediction, and user interface components. The propositioned solution consists of various software components, is developed in the Python programming language, and is implemented in a Google Colab or Jupyter Notebook environment. Data preprocessing and modeling are handled using the following Python libraries: sci-kit learn, Pandas, NumPy,

Matplotlib, Seaborn, and SciPy. Ipywidgets, another Python library, is used to employ the user interface.

The hardware architecture includes appropriate computers for development use that have sufficient CPU, RAM, and GPU resources to support model training, development, and evaluation. Cloud-based resources are used for model computation and data storage. The proposed solution's design encourages scalability, maintainability, and model enhancements.

Functional Requirements

- The system will allow users to input predefined student data points in a user-friendly interface.
- The system will preprocess user input using scaling and encoding methods identical to those applied during model training.
- The trained gradient boosting classifier model will generate one of three prediction outcomes: drop out, remain enrolled, or graduate.
- The system must clearly and innately display the predicted outcome to the user.
- The system will support continual predictions without required restarts.
- The system will have the option to generate an evaluation report that includes accuracy, precision, recall, and F1-score. The system will give a brief overview defining these scores for the user.
- The system must be scalable, supporting model training and updates to incorporate new student data as it becomes available.

Nonfunctional Requirements

- The system will generate predictions within a reasonable time frame after the user provides the requested input and presses the prediction button. This may require hyperparameter values to be hardcoded.
- The model should achieve an overall classification accuracy of approximately 75-80% or higher.
- The system must be capable of consistently predicting the same classification output when identical inputs are entered.
- The system will feature a UI that is easy to navigate for users to enter the required information needed to calculate a prediction.
- Student data will be handled securely so that no personally identifiable information is unveiled or stored.
- The system will be scalable to support larger datasets and the addition of more features to existing datasets.

Flowchart and Use Case Diagram

Figure 4 (flowchart) illustrates the workflow of the student outcome prediction system, showing how the user-provided data is processed and transformed into a predicted classification.

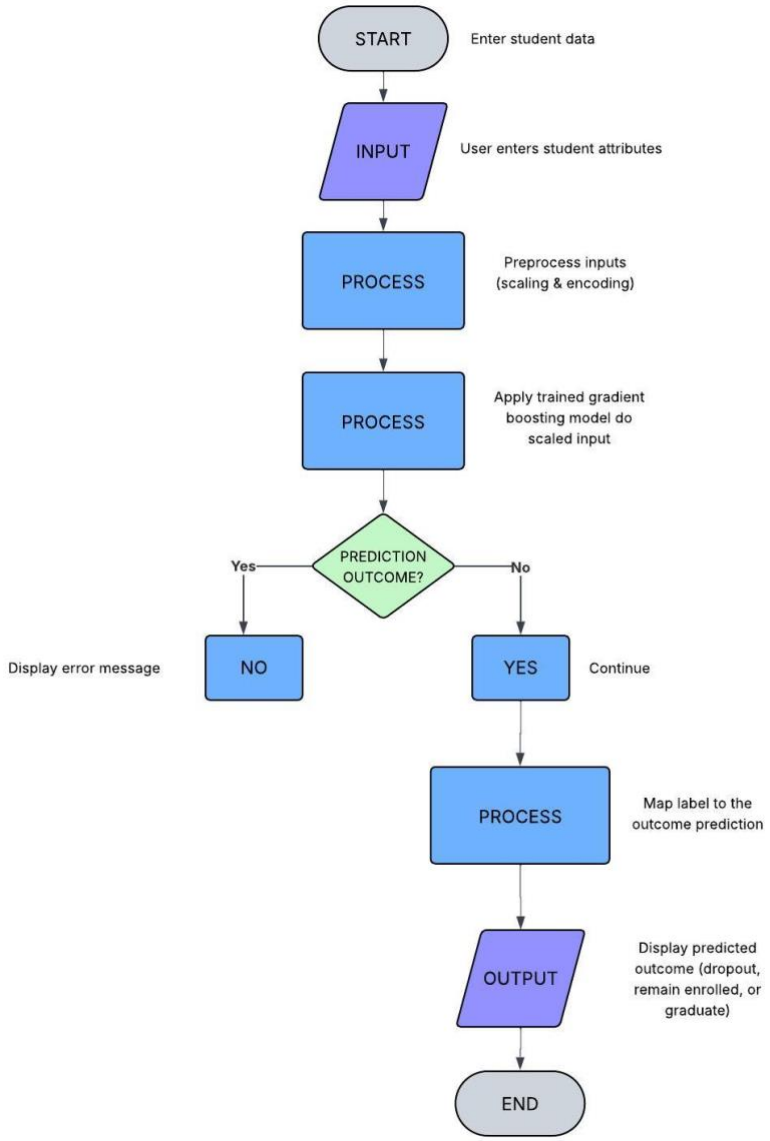

Figure 4: Flowchart

Note. Created using *LucidChart* [Diagram], 2026. LucidChart (https://lucid.app/lucidchart/83bbdde7-b456-48a3-ab18-7a334fafdc7e/edit?viewport_loc=-527%2C-318%2C2204%2C2324%2C0_0&invitationId=inv_1fb77fef-9833-4f26-bce2-4fefa0bacbb0).

Figure 5 (use case diagram) illustrates how university staff and administrators will interact with the prediction system to submit student data, generate success outcome predictions, and review model performance.

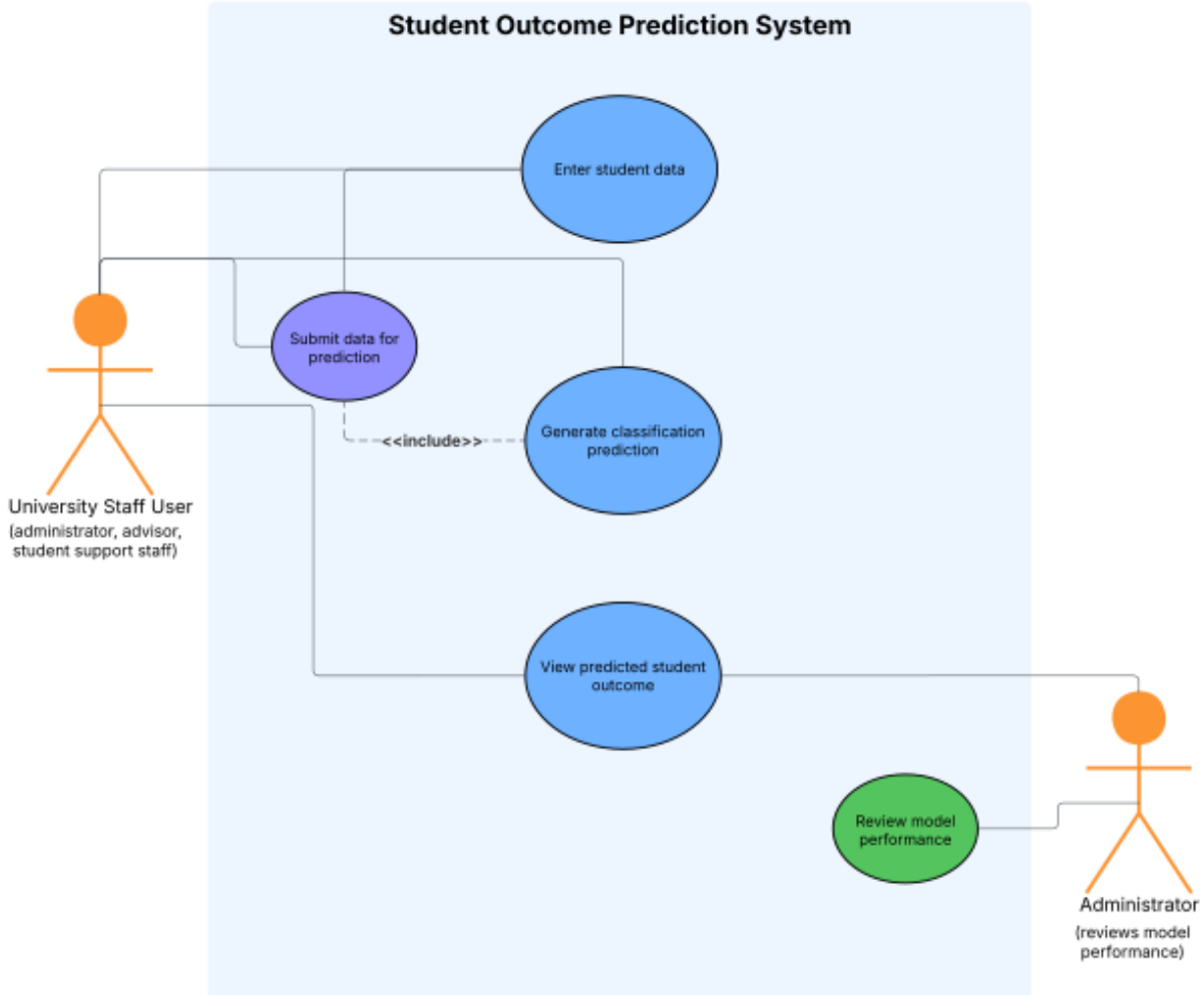

Figure 5: Use case diagram

Note. Created using *LucidChart* [Diagram], 2026. LucidChart (https://lucid.app/lucidchart/83bbdde7-b456-48a3-ab18-7a334fafdc7e/edit?viewport_loc=-527%2C-318%2C2204%2C2324%2C0_0&invitationId=inv_1fb77fef-9833-4f26-bce2-4fefa0bacbb0).

Data Handling

The data used in this product consists of demographic, socioeconomic, and academic performance information collected from first-year college students. The dataset is initially reviewed to assess data quality, identify missing values and inaccuracies, and understand feature distributions. The categorical variables are encoded as numerical values for use in the machine learning algorithm. Numerical features are scaled using Sci-kit Learn's StandardScaler() class to ensure consistency across inputs. The StandardScaler() class calculates a z-score, or standard score, for each feature to transform each numerical value to have a mean of zero and a standard deviation of one, making all values close to zero. All numerical values are standardized to a standard scale to improve the model's predictive capability, as some features have larger numerical ranges, such as the first-semester curricular units (grade), which range from 0 to 19, and categorical variables that are binary (0 or 1), such as scholarship or displacement status. The target variable is encoded because numerical class labels represent the possible prediction outcomes: drop out (0), remain enrolled (1), and graduate (2). These numerical labels are mapped to outcomes in the user interface that are interpretable for humans reading the output. These preprocessing steps are applied during model training and again when processing user input on the final prediction model. This ensures that user inputs are transformed the same way the training data was, before the algorithm calculates a classification prediction.

Curricular units 1st sem (grade)	Curricular units 1st sem (approved)	Curricular units 1st sem (enrolled)	Tuition fees up to date	Scholarship holder	Displaced	Target
0.000000	0	0	1	0	1	Dropout
14.000000	6	6	0	0	1	Graduate
0.000000	0	6	0	0	1	Dropout
13.428571	6	6	1	0	1	Graduate
12.333333	5	6	1	0	0	Graduate
11.857143	5	5	1	0	0	Graduate
13.300000	7	7	1	1	1	Graduate
0.000000	0	5	0	0	1	Dropout
13.875000	6	6	1	1	0	Graduate
11.400000	5	6	0	0	1	Dropout

Figure 6: This shows the features selected for the user interface as they appear in raw, human-readable form. This is how users enter student data into the system before preprocessing and transformation are applied to the values.

Curricular units 1st sem (grade)	Curricular units 1st sem (approved)	Curricular units 1st sem (enrolled)	Tuition fees up to date	Scholarship holder	Displaced	Target
-2.197102	-1.521257	-2.528560	0.368135	-0.574914	0.907512	0
0.693599	0.418050	-0.109105	-2.716392	-0.574914	0.907512	2
-2.197102	-1.521257	-0.109105	-2.716392	-0.574914	0.907512	0
0.575611	0.418050	-0.109105	0.368135	-0.574914	0.907512	2
0.349468	0.094832	-0.109105	0.368135	-0.574914	-1.101914	2
0.251144	0.094832	-0.512348	0.368135	-0.574914	-1.101914	2
0.549064	0.741268	0.294137	0.368135	1.739390	0.907512	2
-2.197102	-1.521257	-0.512348	-2.716392	-0.574914	0.907512	0
0.667789	0.418050	-0.109105	0.368135	1.739390	-1.101914	2
0.156754	0.094832	-0.109105	-2.716392	-0.574914	0.907512	0

Figure 7: This shows the features selected for the user interface after preprocessing and scaling are applied using StandardScaler. These transformed values are used by the gradient boosting classifier model as input and ensure the values are on a consistent scale before the system makes a prediction. This is important because some numerical features, such as first-semester grades, are on a much larger scale than binary features like scholarship holder or tuition status.

Algorithm Functionality

The product uses a gradient boosting classifier model to predict student outcomes from processed input features. Gradient boosting is an ensemble method, meaning it consists of multiple small base learner models that are trained and combined to create a single prediction model. This reduces variance, allowing the model to find patterns in the base learner models with less noise present in the data, which can arise when training on a single simplified model.

The model is trained using an 80/20 test/train split. Stratified k-fold cross-validation is used to improve consistency when working with imbalanced outcome classes, such as those found in this dataset. Randomized search cross-validation is used during model development to investigate hyperparameter values and identify the best-performing denominations for parameters such as learning rate, tree depth, number of estimators, and subsampling rate. These best hyperparameter values are then hardcoded into the final model for increased prediction speed.

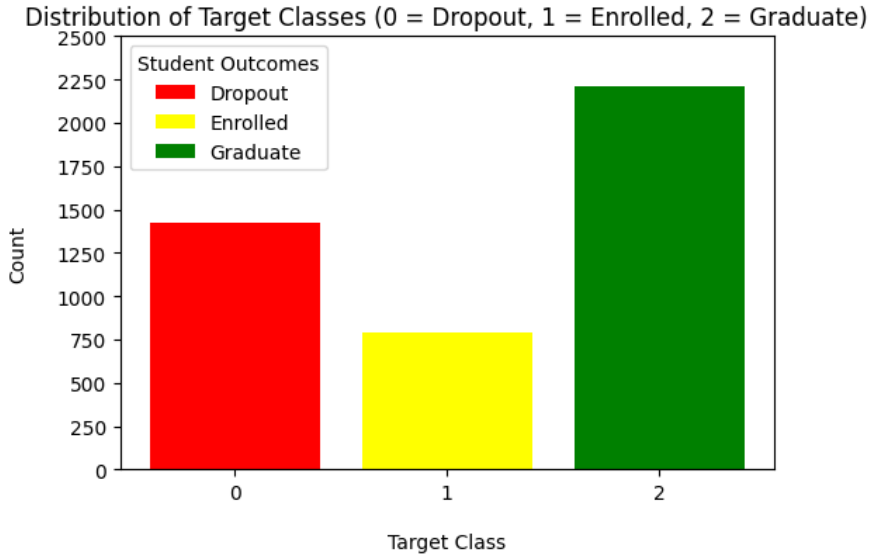

Figure 8: Class imbalance visualized during EDA

The final prediction model uses stochastic subsampling, which trains each decision tree on a random split of the training data. The subsample parameter helps reduce model overfitting by introducing randomness during model training. The optimal value for this parameter is found through randomized search cross-validation. Optimal hyperparameter values will be identified and hardcoded into the final model to provide faster runtime to the user, rather than requiring the user interface to run hyperparameter tuning techniques during the prediction process, which can take several minutes. The trained model will output one of three classification outcomes: drop out, remain enrolled, or graduate.

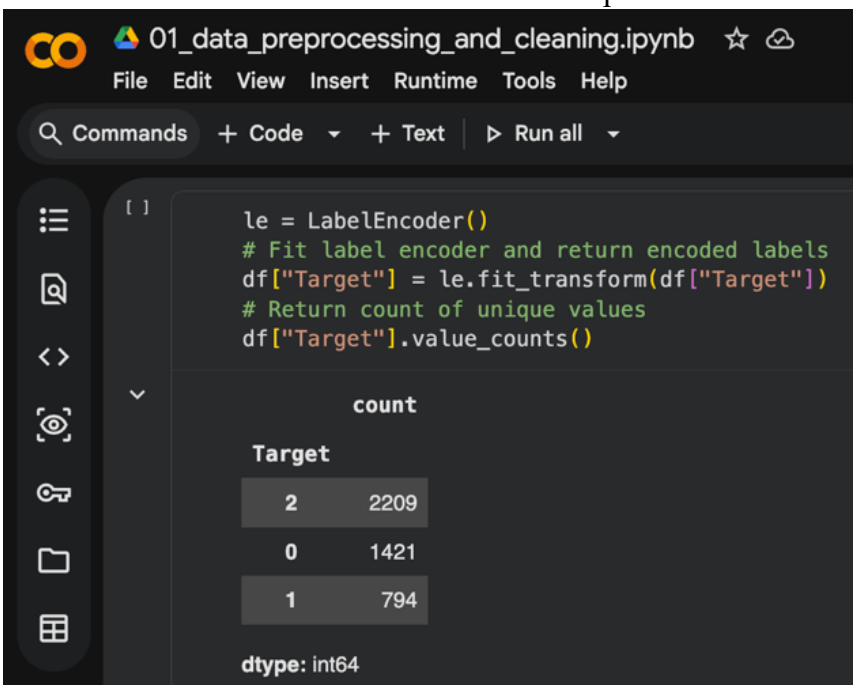

Figure 9: LabelEncoder() used during data preprocessing

```python
# Fix the typo in the raw dataset
df_raw = df_raw.rename(columns={'Nacionality': 'Nationality'})

# Map the Target variable in the raw dataset to its numerical value as found in
the processed dataset
target_variable_map = {
    'Dropout': 0,
    'Enrolled': 1,
    'Graduate': 2
}
if df_raw['Target'].dtype == 'object':
  df_raw['Target'] = df_raw['Target'].map(target_variable_map)

# Split the data into Target and feature variables
X_raw = df_raw.drop(columns=['Target'])
y = df_raw['Target']

# Split raw data into 80/20 train/test split
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X_raw, y, test_size = 0.2, stratify = y, random_state = 42
)

# Fit StandardScaler() on training data
scaler = StandardScaler()
X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train_raw), columns =
X_train_raw.columns)
X_test_scaled = pd.DataFrame(scaler.transform(X_test_raw), columns = X_test_raw.
columns)

# Hard code the best parameters for the model (found using GridSearchCV)
set_best_params = {
    'learning_rate': np.float64(0.1707829063523625),
    'max_depth': 3,
    'n_estimators': 143,
    'subsample': np.float64(0.9480528898228043),
    'random_state': 42
}

final_classifier_model = GradientBoostingClassifier(**set_best_params)
final_classifier_model.fit(X_train_scaled, y_train)

# Print model accuracy
print('Train accuracy:', final_classifier_model.score(X_train_scaled, y_train))
print('Test accuracy:', final_classifier_model.score(X_test_scaled, y_test))
raw_feature_columns = list(X_train_raw.columns)

# Map the classes to labels (0 is dropout, 1 enrolled, 2 graduate)
label_map = {0: 'Dropout', 1: 'Remain Enrolled', 2: 'Graduate'}
```

```python
# Predict students' drop-out risk
def on_predict_button_clicked(b):
    with output:
      # Clear previous output
      output.clear_output()

      # Convert the user input from raw to scaled values
      input_scaled = build_input_row_scaled()

      prediction_class = int(final_classifier_model.predict(input_scaled)[0])
      prediction_label = label_map.get(prediction_class, f'Unknown
      ({prediction_class})')

      print('Predicted Student Outcome:')
      print(f'{prediction_label}')

predict_button.on_click(on_predict_button_clicked)
```

Figure 10: Target classes are mapped to labels when processing user input in the UI

Product Integration

The proposed student outcome prediction system is designed as an analytical tool that consumes data from existing university systems and exports the student data, then loads it into the model as a dataset. The prediction results can be used by academic advisors, administrators, or student support staff to help inform decisions about student support and intervention. Because the system uses a flexible development methodology and contains clearly defined input features with consistent preprocessing steps, it can be updated or expanded as new data becomes available or as the customer's needs change.

Privacy, Security, and Compliance Requirements

The proposed solution is designed to honor privacy and security best practices in handling student data. The system does not require personally identifiable information (PII) to be stored within the prediction interface. All data used for model training and evaluation will be anonymized, and the datasets and model outputs will be restricted to authorized university personnel who will utilize the information and model to make ethical, informed decisions about retention efforts. The system will comply with educational privacy standards, including the Family Educational Rights and Privacy Act (FERPA). Student information will be used only for executive choices that affect the institution's retention programs and services.

Acceptance Criteria

The proposed product will be considered acceptable when the following criteria are met:

- The model achieves a prediction accuracy of approximately 75-80% on the test dataset.

- Users can enter student data and receive a prediction outcome of drop out, remain enrolled, or graduate.
- The prediction system produces consistent predictions for identical inputs.
- The user interface runs without errors during demonstrations and stakeholder reviews.
- The solution meets the defined business objectives and the established Definition of Done.

Testing Approach

The testing approach for the solution focuses on validating model performance and system functionality to certify the system functions as expected and meets the acceptance criteria defined in the Acceptance Criteria section (above).

- Model performance testing will assess the model's predictive performance using an 80/20 test/train split. Accuracy, precision, recall, f1-score, and confusion matrices will be reviewed to verify that the model achieves a prediction accuracy score within the required range of approximately 75-80%.
- Functional testing will assess that the user interface can successfully submit inputs, then scale and transform those input values, and run that data through the predictive algorithm to generate one of three expected prediction outcomes: drop out, remain enrolled, or graduate.
- The system's ability to produce consistent predictions for identical inputs will be tested by submitting multiple identical values numerous times to verify that the model produces the same prediction outcome for the identical input values.

- Usability and stability of the proposed system will be tested during demonstrations with stakeholders to confirm that the interface runs without errors and handles user interactions as expected, without interruption.

Test Case Examples

The following test cases represent example scenarios that will be used to validate the product's accuracy, performance, and functionality:

Test Case 1: Validation of Prediction Accuracy

- Using a labeled test dataset, the model's classification performance will be evaluated on accuracy, precision, recall, and F1-score. Confusion matrices will be reviewed to confirm that the model achieves a prediction accuracy within the required range of approximately 75-80%.

Test Case 2: User Input and Prediction Generation Testing

- To verify that user inputs are accepted and handled correctly, sample student data from the original raw dataset will be entered into the user interface. The system should accept, scale, and transform the input data, then successfully pass these values to the gradient boosting classifier model. This process will be validated by confirming that one of the three expected outcomes is returned and that it matches the status in the Target variable column of the dataset. Some discrepancies will occur, but should not exceed 20-25% of test values, as the expectation is that the model will accurately predict 75-80% of outcomes.

Test Case 3: Testing the Consistency of Predictions

- To test the consistency of the model's prediction abilities, identical input values from the original raw dataset will be submitted multiple times to ensure that the system produces consistent prediction outcomes for the same data.

Test Case 4: Testing UI Stability and Usability

- To test the stability and usability of the prediction system, demonstrations with model prototypes will be held with stakeholders to confirm that the UI runs without errors, responds appropriately to user interactions, and demonstrates an intuitive, easy-to-use design.

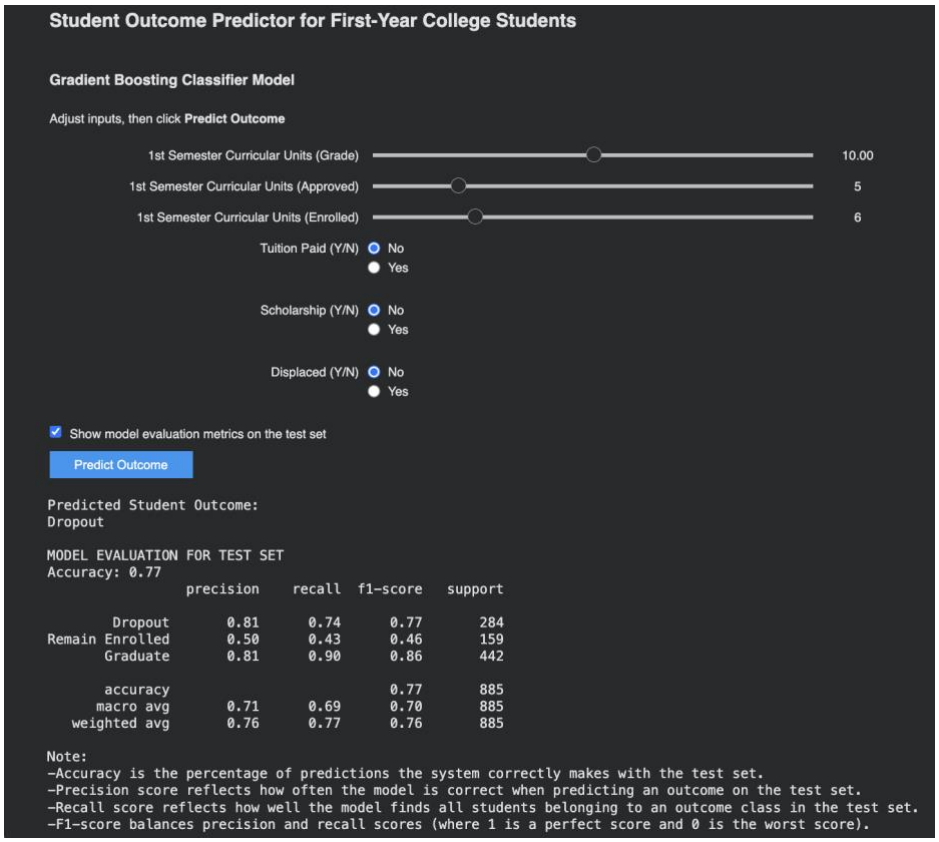

Figure 11: A prototype for the student outcome prediction system user interface

Product Improvement or Enhancement

To enhance the product in the future, a separate prediction interface may be designed specifically for incoming students who have not yet completed any first-semester coursework. This version of the model would use only attributes that are available prior to enrollment, such as demographic and socioeconomic factors, but not academic performance data. This addition would allow universities to flag students who may be at risk before they start their academic studies, enabling their retention initiatives to be more proactive with incoming students.

This product enhancement may not rely on the same gradient boosting ensemble model used in the original system. Multiple models should be tested using the features that most influence the outcome status, without including any features related to the first or second semester in the dataset. These academic performance features would be excluded from this model, and a new dataset containing only socioeconomic and demographic features should be created and scaled. Exploratory data analysis would be performed only on this new, transformed dataset, and various models would be developed and evaluated to determine the best performing model for predicting the outcomes of students who have not yet begun their classes.

Over time, this supplement to the prediction system could be used with ongoing model retraining as more student data becomes available, which would further improve its predictive accuracy and reliability. By expanding the system to support both incoming and current first-year students, the product will provide administrators and student support teams with more data-driven insights to inform retention planning and early intervention initiatives.

Proposal Summary

Conclusion

The proposed project addresses the challenge universities face in identifying first-year college students who may be at risk of dropping out. Identifying these students early is critical for improving retention, but many schools lack data-driven tools that can support timely intervention. The proposed solution is a student outcome prediction system that uses demographic, socioeconomic, and academic performance data to classify students as likely to drop out, remain enrolled, or graduate. By combining machine learning techniques with a simple and accessible user interface, the system translates student data into meaningful outcome predictions that inform the university's advising and retention efforts. This technical approach emphasizes reliability, explainability, and scalability of data while preserving student privacy through complying with federal data protection laws.

The solution uses an Agile development approach with the Scrum framework, allowing the model and user interface to be honed through iterative sprints and stakeholder feedback. A gradient boosting classification model is developed and evaluated using established performance metrics, with a target accuracy of approximately 75-80%. The project is designed to be completed within 6-8 weeks and includes model development, testing, and deployment for demonstration purposes. The successful implementation of this solution will provide administrators and student support staff with a practical tool to support data-informed retention strategies and early intervention efforts.

Stakeholders are asked to approve proceeding with the proposed project implementation, including allocation of resources required for model development, testing, and stakeholder demonstrations. Approving this project will allow the team to move forward with development and evaluation, with the goal of delivering a functional predication system that improves student success outcomes and offers a proactive data-driven approach to student support planning for first-year college students.

Link to GitLab

Link to GitLab D683 Project (Updated with notebook 00 for D687)

Links to Google Colab notebooks

00_run_in_colab.ipynb

01_data_preprocessing_and_clearning.ipynb

02_exploratory_data_analysis.ipynb

03_logistic_regression_model.ipynb

04_gradient_boosting_model.ipynb

05_model_comparison_and_conclusions.ipynb

References

Ab Rahman, N.F., Wang, S.L., & Khalid, N. (2025). Ensemble learning in educational data analysis for improved prediction of student performance: A literature review. *International Journal of Modern Education, 7*(24), 887-902. https://doi.org/10.35631/IJMOE.724064

Apunmayta, R. Q., Ataypoma, D.H., Cayllahua, J.C., Choque, V.I., Pari, A.C., & Valverde, J.C.C. (2024). University drop-out: A systematic review of the main determinant factors (2020-2024) [version 2; peer review: 2 approved]. *F1000Research* 2024, *13:942*. https://doi.org/10.12688/f1000research.154263.2

Baptista, L., Machado, J., Monica, M., & Realinho, V. (2021). *Predict students' drop-out and academic success: Investigating the impact of social and economic factors* [Data set]. Kaggle. https://www.kaggle.com/datasets/thedevastator/higher-education-predictors-of-student-retention

Farhood, H., Joudah, I., Beheshti, A., & Muller, S. (2024). Evaluating and enhancing artificial intelligence models for predicting student learning outcomes. *Informatics*, *11*(3), 46. https://doi.org/10.3390/informatics11030046

GeeksforGeeks. (2025). CatBoost in machine learning. https://geeksforgeeks.org/machine-learning/catboost-ml/

GeeksforGeeks. (2025). Ways to import CSV files in Google Colab. https://www.geeksforgeeks.org/machine-learning/ways-to-import-csv-files-in-google-colab/

Guzman Rincon, A., Barragan Moreno, S., & Cala-Vitery, F. (2021). Rural population and COVID-19: A model for assessing the economic effects of drop-out in higher education. *Frontiers in Education, 6,* Article 812114. https://doi.org/10.3389/feduc.2021.812114

Hanson, M. (2025). College drop-out rates. EducationData.org. https://educationdata.org/college-dropout-rates

Hygger. (n.d.). *Sprints: All you need to know about sprints in project management* [Image]. https://hygger.io/guides/agile/scrum/sprints-in-scrum/

Lucidchart. (2025). *Lucidchart* [Diagramming software]. https://lucid.app/

Nexapp. (2024). *Agile software development: Everything you need to know* [Image]. https://www.nexapp.ca/en/blog/agile-software-development

Paul VII. (2017). *Scrum master certification: PSM 1 exam: Preparation guide and handbook* [Audiobook]. Pashun Consulting.

Project Jupyter. (2023). Jupyter widgets 8.1.8 documentation: How-to guides. *https://ipywidgets.readthedocs.io/en/latest/how-to/*

Scikit-learn developers. (2025). User guide. Scikit-learn documentation. https://scikit-learn.org/stable/user_guide

Stephens, R. (2023). *Beginning software engineering* (2nd ed.). Wiley.

Villar, A., & Velini de Andrade, C.R. (2024). Supervised machine learning algorithms for predicting student dropout and academic success: A comparative study. *Discover Artificial Intelligence, 4,* Article 2. https://doi.org/10.1007/s44163-023-00079-z

Visual Paradigm. (2025). *Why fixed length sprints in scrum?* [Image]. https://www.visual-paradigm.com/scrum/why-fixed-length-of-sprints-in-scrum/